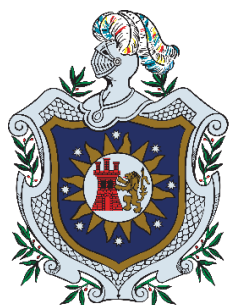


Universidad Nacional Autónoma de Nicaragua, León

Área de Conocimiento de

Área Específica de



PORTAFOLIO PROYECTO APP PARA WEB EN INGLÉS

React SPA, componentes y navegación + Formularios, validación y servicios API con
Laravel

Elaborado por:

- Julio Cesar Maradiaga Huete
- Sebastian Sebastian Antonio Trujillo Zepeda
- Genesis Tatiana Narváez
- Josnar Efraín Cruz Romero
- Franvir Steven Rivas Vázquez

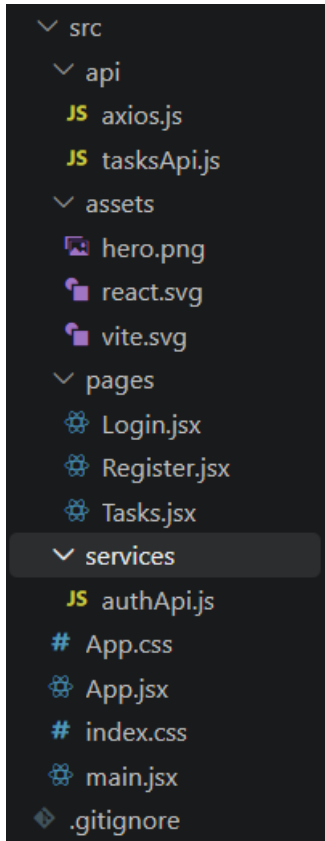
Docente o tutor: Aldo Berrios

Fecha: 24-09-26

www.unanleon.edu.ni

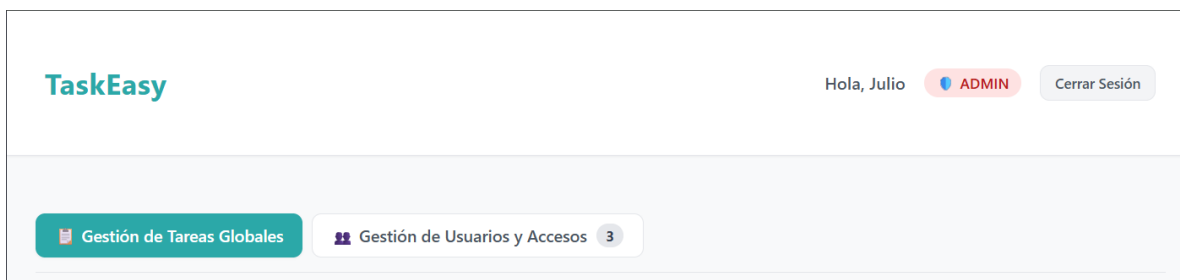
2025: 46/19 ¡Siempre más allá, Seguimos Avanzando en la Revolución!

Organizada modularmente separando componentes de UI, páginas principales (Tasks.jsx, Login.jsx, Register.jsx) y servicios de comunicación HTTP (api/axios.js).

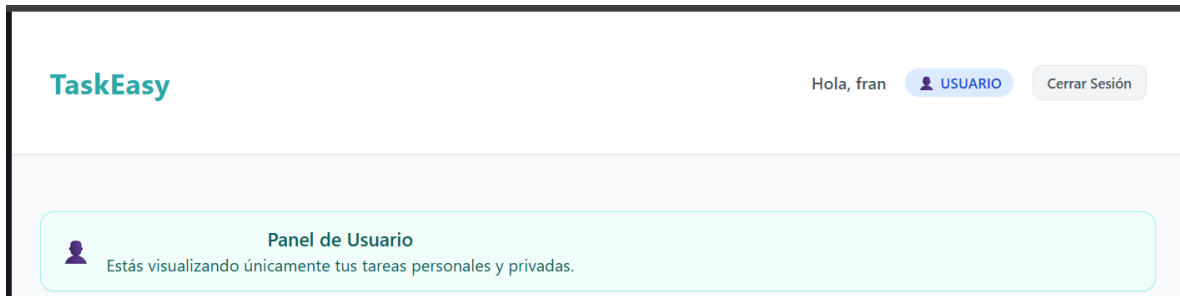


Implementación de una barra de navegación superior dinámica (Header) que adapta los permisos visuales y accesos según el rol del usuario autenticado (Administrador o Usuario Regular).

NavBar del usuario administrador



NavBar del usuario común



Rutas de autenticación completamente segregadas de las vistas protegidas por tokens de sesión y verificación de roles (user, admin, blocked).

Pestañas de control administrativo interactivas mediante estados de React para alternar ágilmente entre la Gestión de Tareas Globales y la Gestión de Usuarios y Accesos.

Control de Usuarios Registrados y Estadísticas				Gestión de permisos y accesos	
USUARIO	CORREO ELECTRÓNICO	TAREAS CREADAS	ROL ACTUAL	ACCIONES Y ACCESOS	
Julio	julymaradiaga3@gmail.com	2 tareas	ADMIN	Administrador	Eliminar
sebas	st26242@gmail.com	5 tareas	USER	Usuario	Eliminar
fran	franvirestiven@gmail.com	1 tareas	USER	Administrador Bloqueado	Eliminar

El backend filtra estrictamente los recursos mediante políticas y validación de roles en los controladores (TaskController y UserController), asegurando que un usuario regular solo manipule sus propios datos, mientras que el administrador posee facultades globales.

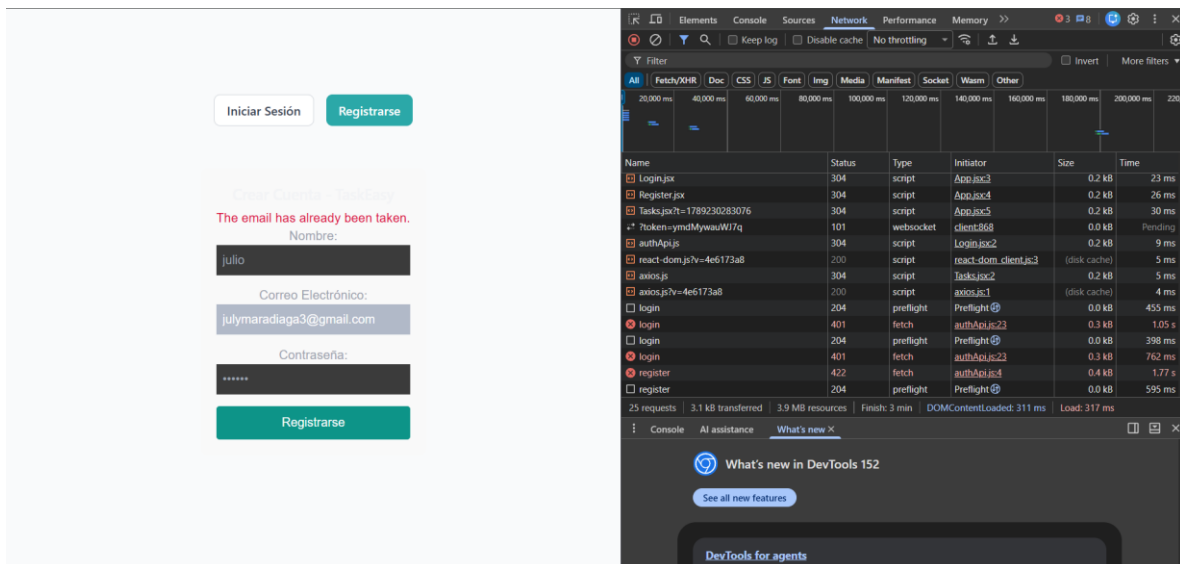
Operaciones de creación, lectura, actualización y eliminación modularizadas con retroalimentación visual inmediata.

Gestión reactiva del estado local para campos de texto y selectores de prioridad (Baja, Media, Alta).

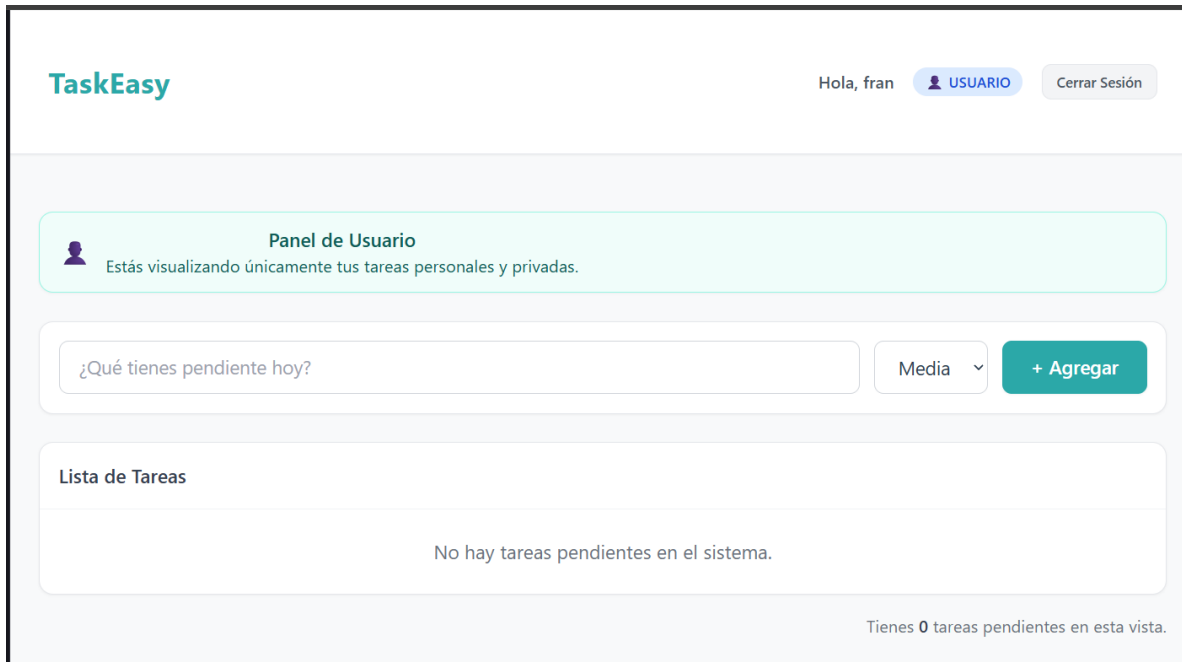


control de errores de validación tanto en el registro, inicio de sesión como en la manipulación de datos del sistema.

Cuando el usuario interactúa con un formulario o realiza una acción en la interfaz, el componente dispara una solicitud HTTP asíncrona mediante Axios, adjuntando automáticamente el token Bearer de autenticación almacenado en el navegador.

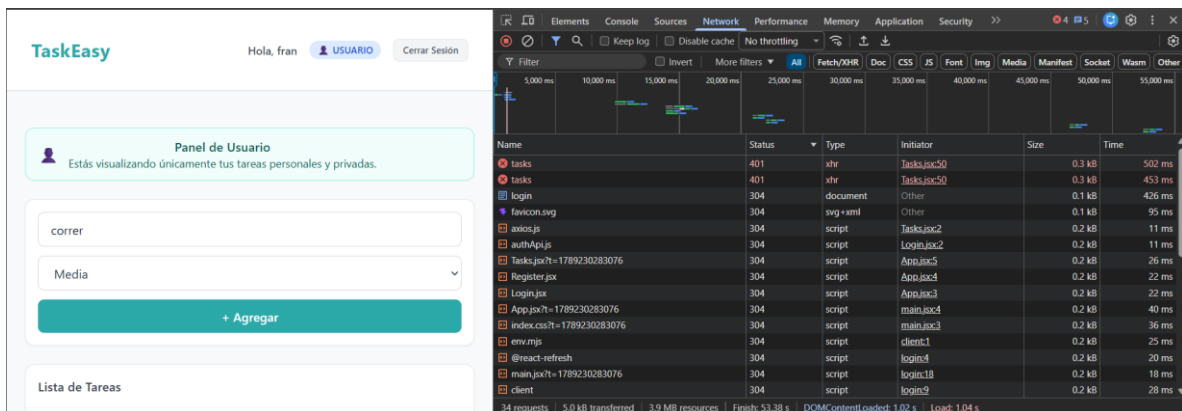


Manejo de estados de carga (loading), renderizado condicional para listas vacías



Control de respuestas 401 (No autorizado), 403 (Prohibido/Roles), 404 (No encontrado), 422 (Error de validación) y el flujo del camino exitoso.

La petición llega al archivo de rutas de la API de Laravel, donde el middleware auth:sanctum valida las credenciales y el token del usuario.



TaskEasy

Hola, fran

USUARIO

Cerrar Sesión

Panel de Usuario

Estás visualizando únicamente tus tareas personales y privadas.

¿Qué tienes pendiente hoy?

Media

+ Agregar

Lista de Tareas

Elements

Console

Sources

Network

Performance

Memory

Application

Security

Keep log

Disable cache

No throttling

Filter

Invert

More filters

All

Fetch/XHR

Doc

CSS

JS

Font

Img

Media

Manifest

Socket

Warm

Other

Name	Status	Type	Initiator	Size	Time
users	403	fetch	VM2507:1	0.3 kB	695 ms
login	304	document	Other	0.1 kB	19 ms
favicon.svg	304	svg+xml	Other	0.1 kB	7 ms
axios.js	304	script	Tasks.jsx2	0.2 kB	13 ms
authApi.js	304	script	Login.jsx2	0.2 kB	12 ms
Tasks.jsx?i=1789230283076	304	script	App.jsx5	0.2 kB	22 ms
Register.jsx	304	script	App.jsx4	0.2 kB	19 ms
Login.jsx	304	script	App.jsx3	0.2 kB	16 ms
App.jsx?i=1789230283076	304	script	main.jsx4	0.2 kB	21 ms
index.css?i=1789230283076	304	script	main.jsx3	0.2 kB	22 ms
env.rn.js	304	script	client.1	0.2 kB	15 ms
@react-refresh	304	script	login4	0.2 kB	14 ms
main.jsx?i=1789230283076	304	script	login18	0.2 kB	15 ms
client	304	script	login9	0.2 kB	22 ms
tasks	204	preflight	Preflight@	0.0 kB	560 ms

28 requests | 4.3 kB transferred | 3.9 MB resources | Finish: 6.5 min | DOMContentLoaded: 411 ms | Load: 419 ms

TaskEasy

Hola, fran

USUARIO

Cerrar Sesión

Panel de Usuario

Estás visualizando únicamente tus tareas personales y privadas.

¿Qué tienes pendiente hoy?

Media

+ Agregar

Lista de Tareas

Elements

Console

Sources

Network

Performance

Memory

Application

Security

Keep log

Disable cache

No throttling

Filter

Invert

More filters

All

Fetch/XHR

Doc

CSS

JS

Font

Img

Media

Manifest

Socket

Warm

Other

Name	Status	Type	Initiator	Size	Time
recurso-que-no-existe	404	fetch	VM2508:1	10.3 kB	882 ms
login	304	document	Other	0.1 kB	39 ms
favicon.svg	304	svg+xml	Other	0.1 kB	10 ms
axios.js	304	script	Tasks.jsx2	0.2 kB	5 ms
authApi.js	304	script	Login.jsx2	0.2 kB	16 ms
Tasks.jsx?i=1789230283076	304	script	App.jsx5	0.2 kB	27 ms
Register.jsx	304	script	App.jsx4	0.2 kB	13 ms
Login.jsx	304	script	App.jsx3	0.2 kB	9 ms
App.jsx?i=1789230283076	304	script	main.jsx4	0.2 kB	75 ms
index.css?i=1789230283076	304	script	main.jsx3	0.2 kB	78 ms
env.rn.js	304	script	client.1	0.2 kB	78 ms
@react-refresh	304	script	login4	0.2 kB	76 ms
main.jsx?i=1789230283076	304	script	login18	0.2 kB	12 ms
client	304	script	login9	0.2 kB	41 ms
tasks	204	preflight	Preflight@	0.0 kB	671 ms

26 requests | 13.8 kB transferred | 3.9 MB resources | Finish: 14.46 s | DOMContentLoaded: 486 ms | Load: 493 ms

Laravel empaqueta el resultado en formato JSON junto con su código de estado HTTP correspondiente (como 200, 201 o errores controlados como 422). React recibe esta respuesta, actualiza el estado local de los componentes y refresca la interfaz visual de manera inmediata.

TaskEasy

Hola, fran

USUARIO

Cerrar Sesión

Panel de Usuario

Estás visualizando únicamente tus tareas personales y privadas.

¿Qué tienes pendiente hoy?

Media

+ Agregar

Lista de Tareas

Elements

Console

Sources

Network

Performance

Memory

Application

Security

Keep log

Disable cache

No throttling

Filter

Invert

More filters

All

Fetch/XHR

Doc

CSS

JS

Font

Img

Media

Manifest

Socket

Warm

Other

Name	Status	Type	Initiator	Size	Time
register	422	fetch	VM2560:1	0.5 kB	947 ms
login	304	document	Other	0.1 kB	63 ms
favicon.svg	304	svg+xml	Other	0.1 kB	92 ms
axios.js	304	script	Tasks.jsx2	0.2 kB	6 ms
authApi.js	304	script	Login.jsx2	0.2 kB	5 ms
Tasks.jsx?i=1789230283076	304	script	App.jsx5	0.2 kB	8 ms
Register.jsx	304	script	App.jsx4	0.2 kB	8 ms
Login.jsx	304	script	App.jsx3	0.2 kB	7 ms
App.jsx?i=1789230283076	304	script	main.jsx4	0.2 kB	358 ms
index.css?i=1789230283076	304	script	main.jsx3	0.2 kB	356 ms
env.rn.js	304	script	client.1	0.2 kB	358 ms
@react-refresh	304	script	login4	0.2 kB	357 ms
main.jsx?i=1789230283076	304	script	login18	0.2 kB	16 ms
client	304	script	login9	0.2 kB	15 ms
tasks	204	preflight	Preflight@	0.0 kB	673 ms

26 requests | 3.9 kB transferred | 3.9 MB resources | Finish: 6.03 s | DOMContentLoaded: 826 ms | Load: 831 ms

PREGUNTAS OBLIGATORIAS

1. Explique el recorrido completo de los datos cuando un usuario guarda un registro: desde el formulario en React hasta la base de datos y la respuesta que vuelve a la interfaz.

Cuando el usuario completa el formulario y hace clic en guardar, el componente de React intercepta el evento, previene el comportamiento por defecto y ejecuta una petición asíncrona mediante la capa de servicios (utilizando fetch o Axios), adjuntando en los encabezados el token Bearer de autenticación y los datos en formato JSON.

Recepción y Validación

La petición HTTP llega a las rutas protegidas de la API en Laravel, donde el middleware auth:sanctum valida el token. A continuación, el controlador recibe la solicitud y ejecuta las reglas de validación (por ejemplo, verificando campos obligatorios y llaves foráneas). Si falla, retorna un código 422; si es correcto, el modelo Eloquent procesa los datos y realiza la inserción en la base de datos MySQL.

Laravel empaqueta el nuevo registro creado junto con un código de estado 201 (Created) y lo envía de regreso en formato JSON. React recibe esta respuesta, actualiza el estado local de los componentes y refresca la interfaz visual en tiempo real para mostrar el nuevo elemento.

2. ¿Cómo implementaron la autenticación y cómo se mantiene el acceso del usuario a las rutas privadas?

En el backend se configuró Laravel Sanctum para emitir tokens de acceso personal (createToken) al momento de que el usuario envía credenciales válidas en el endpoint de inicio de sesión.

Mantenimiento en el Frontend:

Al autenticarse con éxito, la SPA almacena el token devuelto por la API en el almacenamiento local del navegador (localStorage). Para cada subsecuente petición a rutas protegidas, la capa de servicios adjunta automáticamente este token en la cabecera Authorization: Bearer <token>. Las rutas de React validan la existencia de este token o del usuario autenticado en el estado global/contexto para permitir o denegar el renderizado de las vistas privadas.

3. ¿Cómo funcionan los roles y permisos en su proyecto? Indique dónde se verifica realmente una acción restringida y por qué no basta con ocultar un botón en React.

La seguridad real se procesa en los controladores y middlewares de Laravel. Por ejemplo, en los endpoints administrativos (como /api/admin/users), el código evalúa

explícitamente si `$request->user()->role === 'admin'`. Si no cumple con la condición, el servidor interrumpe la ejecución y emite una respuesta 403 Forbidden.

Por qué no basta con ocultar botones en React:

Ocultar un componente visual o un botón en la interfaz de React solo evita que un usuario común interactúe gráficamente con él de forma ordinaria, pero la ruta de la API sigue estando expuesta públicamente. Si un usuario malintencionado realiza una petición HTTP directa (por ejemplo, vía consola con `fetch`), el backend es el único responsable de bloquear el acceso real a los datos y proteger la integridad del sistema.

4. Describa una relación entre dos tablas de su proyecto. Explique la llave foránea y las relaciones Eloquent utilizadas.

En TaskEasy se estructuró una relación lógica donde una entidad principal de tareas o registros se conecta con una tabla relacional o catálogo.

Llave foránea y Eloquent:

En la migración de la tabla principal se definió una columna con una llave foránea (`foreignId('categoria_id')` o equivalente) enlazada mediante `constrained()` con su respectiva tabla de origen. En los modelos de Eloquent, esta relación se declaró utilizando los métodos `hasMany()` en el modelo padre y `belongsTo()` en el modelo hijo, permitiendo realizar consultas eficientes con el método `with()` para traer los datos relacionados sin duplicar información.

5. Explique cómo maneja su aplicación los errores 401, 403, 404 y 422. Dé al menos un ejemplo real de su proyecto.

401 (No autorizado): Se dispara cuando se intenta consumir un endpoint protegido sin enviar un token válido o cuando este ha expirado; Laravel intercepta la petición y Sanctum deniega el acceso.

403 (Prohibido): Ocurre cuando un usuario con rol regular intenta acceder a las rutas de administración, activando la validación del controlador que bloquea la acción.

404 (No encontrado): Se genera al consultar un recurso o endpoint que no existe en el sistema (por ejemplo, una URL de API mal escrita o un ID inexistente buscado con `findOrFail()`).

422 (Error de validación): Se produce al enviar formularios con campos vacíos o datos incorrectos (como un correo duplicado), devolviendo los errores específicos que la interfaz React captura para mostrarlos junto a cada campo.

6. Describa un problema técnico importante que encontró el grupo durante el desarrollo, cómo identificaron la causa y qué solución aplicaron.

Durante la integración de los formularios de la SPA con el backend de Laravel, las peticiones POST de registro y creación devolvían un error de rechazo o fallaban al no estructurar correctamente las cabeceras HTTP de contenido y aceptación JSON.

Al revisar la pestaña Network (Red) de las Herramientas de Desarrollador del navegador, se detectó que el servidor respondía con errores de formato o validación inesperados porque faltaban las cabeceras Content-Type: application/json y Accept: application/json. La solución consistió en estandarizar la capa de servicios centralizada en React para incluir siempre dichas cabeceras y manejar de forma unificada la respuesta response.ok y los códigos de estado HTTP.